

番外実験：Arduino

本テキストは2026年5月時点の情報で構成されている。担当者は必要に応じて本テキストを更新すること。

本実験に関わる文書のライセンスはCC0 1.0 Universalである。担当者にはライセンスの変更も含めた全ての編集を認める。

概要

Arduinoは安価で扱いやすい代表的なマイコンボードの一つである。本実験では、Arduinoを用いてLEDを点滅させるプログラム、俗に「Lチカ」と呼ばれるものから始める。自主実験への応用を見据え、最終的にはセンサの値をPCに記録するデータロガーとしての利用を習得することを目標とする。

注意事項

ArduinoはC言語やC++をベースとしている。1年後期の基礎プログラミング演習でC言語を学んでいる場合は復習しつつ読んでいくとよい。

プログラミング未習得の場合は、以降の内容を授業時間内に理解することは難しいため、参考になるWebページの章で挙げた資料を参考に、事前にC言語の基礎を学んでおくといよい。ただし、書籍はバージョンなど細かい違いで混乱する可能性が高いため非推奨である。

環境構築

初心者向け: Arduino IDEのインストール

<https://www.arduino.cc/en/software/>

開発に必要な機能が一通り揃っていて、GUIで簡単に設定ができる。以降の説明はこのソフトを使っている前提で進める。ある程度こうした開発に慣れている場合は後述のCLIやVSCodeの拡張機能を使ってもよい。

Bring Your Projects to Life with Arduino Software



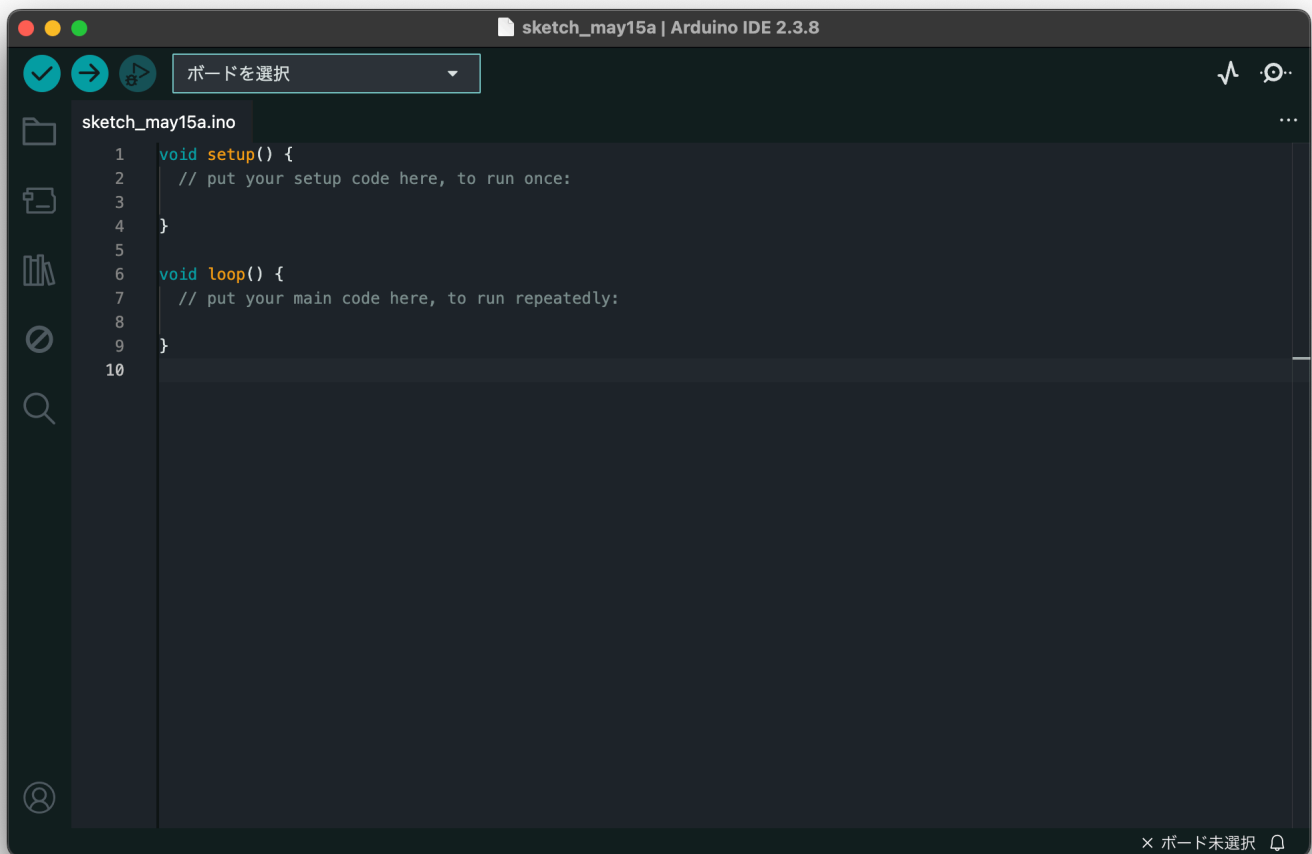
Arduino IDE 2.3.8 [Release notes](#)

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

macOS Apple Silicon 11 Big Sur or newer (64-bit)

DOWNLOAD

インストールは通常のソフトと同様に行う。
完了したら起動する。



ラボ外での確認方法：シミュレータ

<https://www.tinkercad.com/>

TinkercadはAutodesk社が提供する無料のオンライン3Dデザインツールである。電子工作のシミュレーション機能も備わっており、Arduinoをはじめとして様々な部品が使える。

上級者向け1: Arduino CLI

<https://arduino.github.io/arduino-cli/installation/>

上級者向け2: PlatformIO

VSCode拡張機能

<https://marketplace.visualstudio.com/items?itemName=platformio.platformio-ide>

Arduino IDEの設定

スケッチブックの場所

Arduinoのプログラムを保存する場所を指定する。

エディター言語

言語を日本語にすることができる。



コンパイラの警告

どれくらい厳しくエラーを出力するかを指定する。

デフォルト に設定するとよい。

書き込み後にコードを検証する

ONにする。

自動保存

勝手に上書きされたくなければOFF、保存し忘れを防止したければON。

エディターのクイックサジェスト

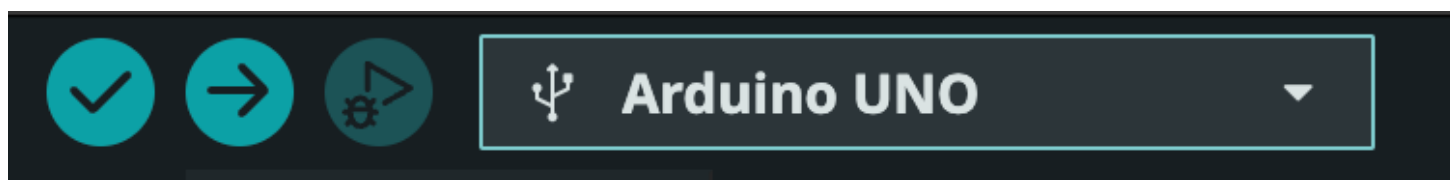
書いている途中で候補が出てくる機能。

ONにすると便利。

実験手順

PCとArduinoの接続

USBケーブル(A to B)でPCとArduinoを接続し、Arduino IDEでボードを選択する。



→(右矢印)を押すとコードをArduinoに書き込む。

☑(チェックマーク)では書き込まずにコードの文法チェックのみを行う。

Arduinoプログラミングの基本

```
void setup() {  
    // スラッシュ二つを先頭に書いた行はコメントアウトになる。プログラムの挙動には影響しないメモ書きなどい  
    // setup関数はプログラムの起動時（リセット時）に一度だけ実行される。  
    // 主な用途はピンの設定など。  
}  
  
void loop() {  
    // loop関数はsetup関数が終わった後に何度も繰り返し実行される。Arduinoが止まらない限り動作し続ける  
}
```

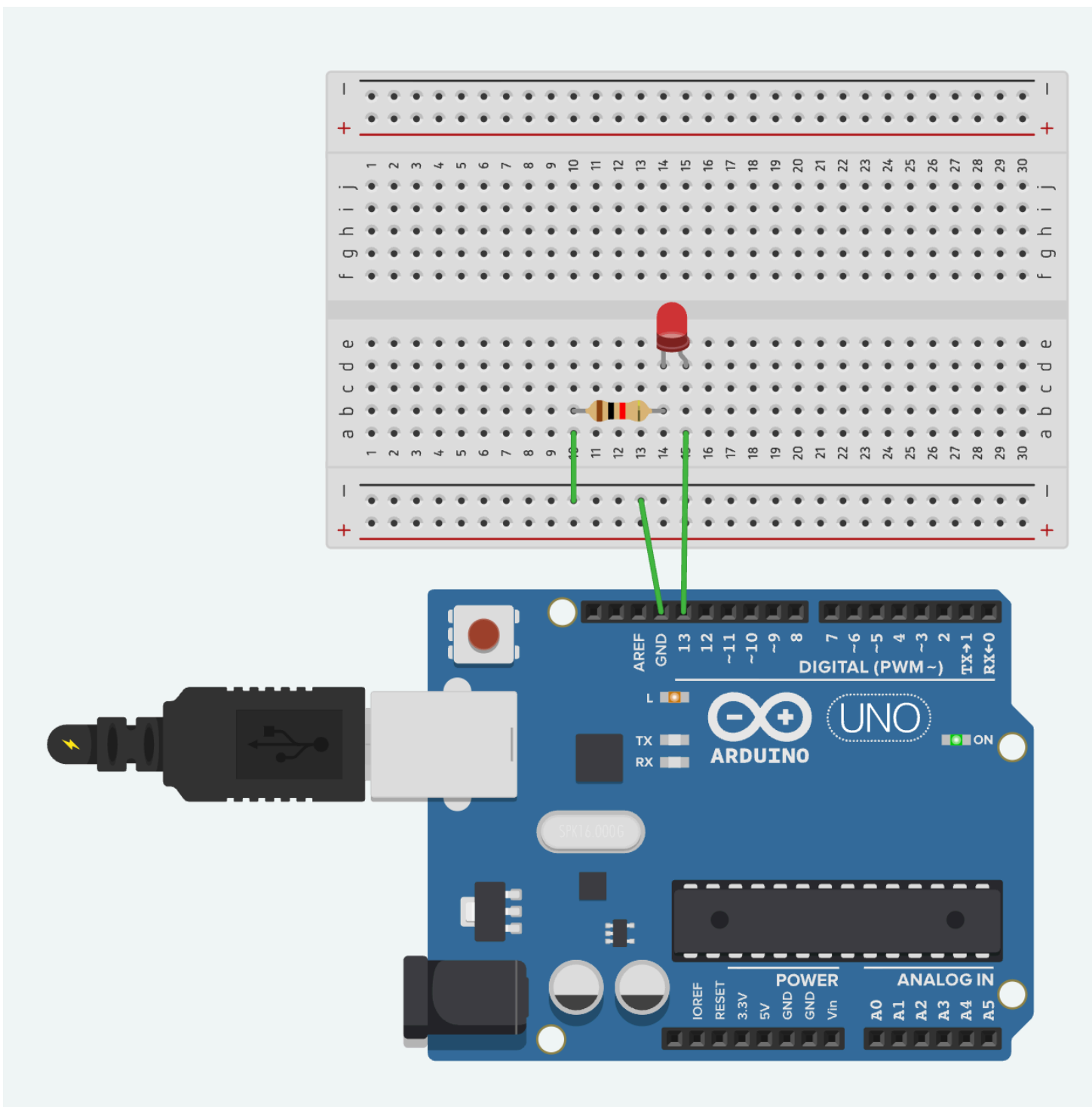
この `void setup()` と `void loop()` という関数の定義だけを行い、具体的な動作を何も指定しないコードを書き込む(右矢印のマークをクリック)。

完了しても何も起こらないが、正常である。IDE下部の出力にエラーが出ていないことを確認する。

Lチカ：LEDを点滅させる

ArduinoにはGPIOという外部との接続ポートが多数備わっている。これを使ってLEDを点滅させる「Lチカ」はArduino学習における"Hello World"的立ち位置のプログラムである。

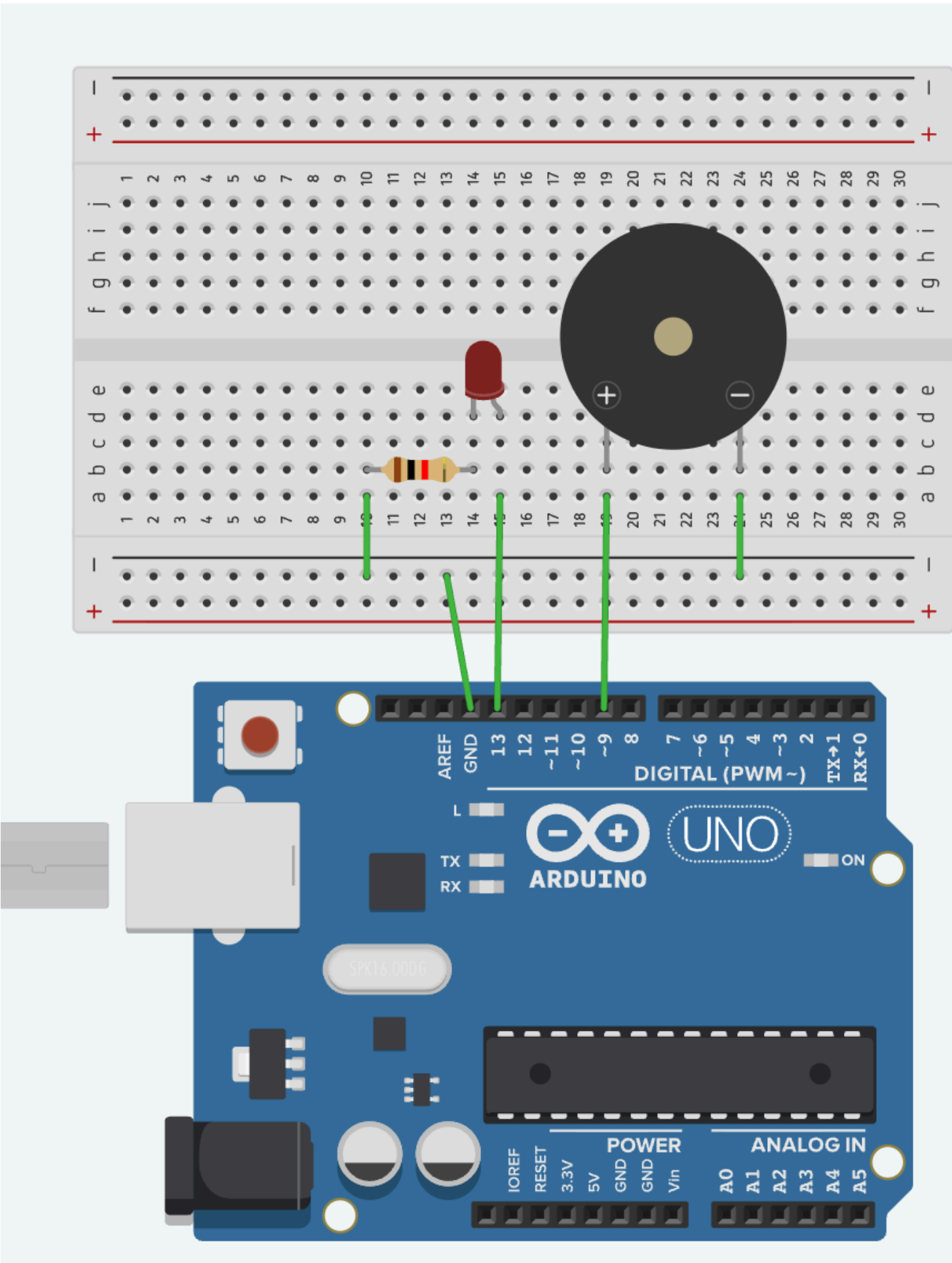
```
void setup() {  
    //13ピンをLEDのアノード(+)に、GNDをカソード(-)に接続する。  
    //後のため、ブレッドボードを使うとよい。  
    pinMode(13, OUTPUT); //13ピンを出力に設定する  
    //このsetup関数内はプログラムの起動時（リセット時）に一度だけ実行される。  
}  
  
void loop() {  
    digitalWrite(13, HIGH); //13ピンをHIGH(5V)にする  
    delay(1000); //1000ミリ秒=1秒待つ  
    digitalWrite(13, LOW); //13ピンをLOW(0V)にする  
    delay(1000); //1000ミリ秒=1秒待つ  
    //このloop関数内を何度も繰り返し続けるため、LEDが点滅する。  
}
```



シミュレータ: <https://www.tinkercad.com/things/1f08fL2G9yL-ledblink?sharecode=t2eqbIRQHKJGVleTDEv0rqaDbygKTUhLPxp3QmmWi3Y>

スピーカーを鳴らす

```
void setup() {  
    //スピーカーを9ピンとGNDに接続する  
}  
  
void loop() {  
    //440Hzの矩形波を9ピンから1秒間出力する  
    tone(9, 440);  
    delay(1000); //1秒待つ  
    noTone(9); // 9ピンのtoneを止める  
    delay(1000); //1秒待つ  
    //これを繰り返し、音が1秒おきになる。  
}
```



シミュレータ: https://www.tinkercad.com/things/5FKkmlv4qLM-beep?sharecode=TjgQm_jloLpmv3qW9PJEsGAD2oYbOyaW7RovpZckFI

PWM制御

```
void setup(){
  pinMode(13, OUTPUT); //LED 13ピンを出力に設定する
  pinMode(9, OUTPUT); //スピーカー 9ピンを出力に設定する
}

void loop(){
  //0-255の範囲でデューティ比(ONの時間の割合)を指定できる。これによりLEDの明るさや、スピーカーの音色
  analogWrite(13, 128);
  analogWrite(9, 128);
  delay(1000);
}
```

回路はそのまま。

信号のONとOFFを繰り返す矩形波において、ONの時間の割合をデューティ比という。これを調整することでLEDが駆動するのに必要な電圧を維持しながら明るさを下げることができる。

[PWM制御とは？原理や使用例をわかりやすく解説 株式会社ニッポー](#)

シミュレータ: https://www.tinkercad.com/things/6KN3Hap0hrG-pwm?sharecode=DwLKYMdQLybGlkkcCv3qyVlb2QsaT1SyTqdUDB5_bDQ

センサー・シリアル通信

超音波センサーは発信した超音波が物体に反射して戻ってくるまでの時間を計測することで、その物体までの距離を計測することができる。

```
//超音波センサー（HC-SR04）での距離を計測し、シリアルモニタに出力する
// VCC → 5V
// GND → GND
// Trig → 12
// Echo → 11

int trigPin = 12; // このように変数を使うとコードが読みやすくなる。今回は12ピンをTriggerに設定して
int echoPin = 11; // 11ピンをEchoに設定
long duration, cm; //距離を格納する変数

void setup() {
    Serial.begin(9600); //シリアル通信を開始する
    pinMode(trigPin, OUTPUT); //トリガーピンを出力に設定する
    pinMode(echoPin, INPUT); //エコーピンを入力に設定する
}

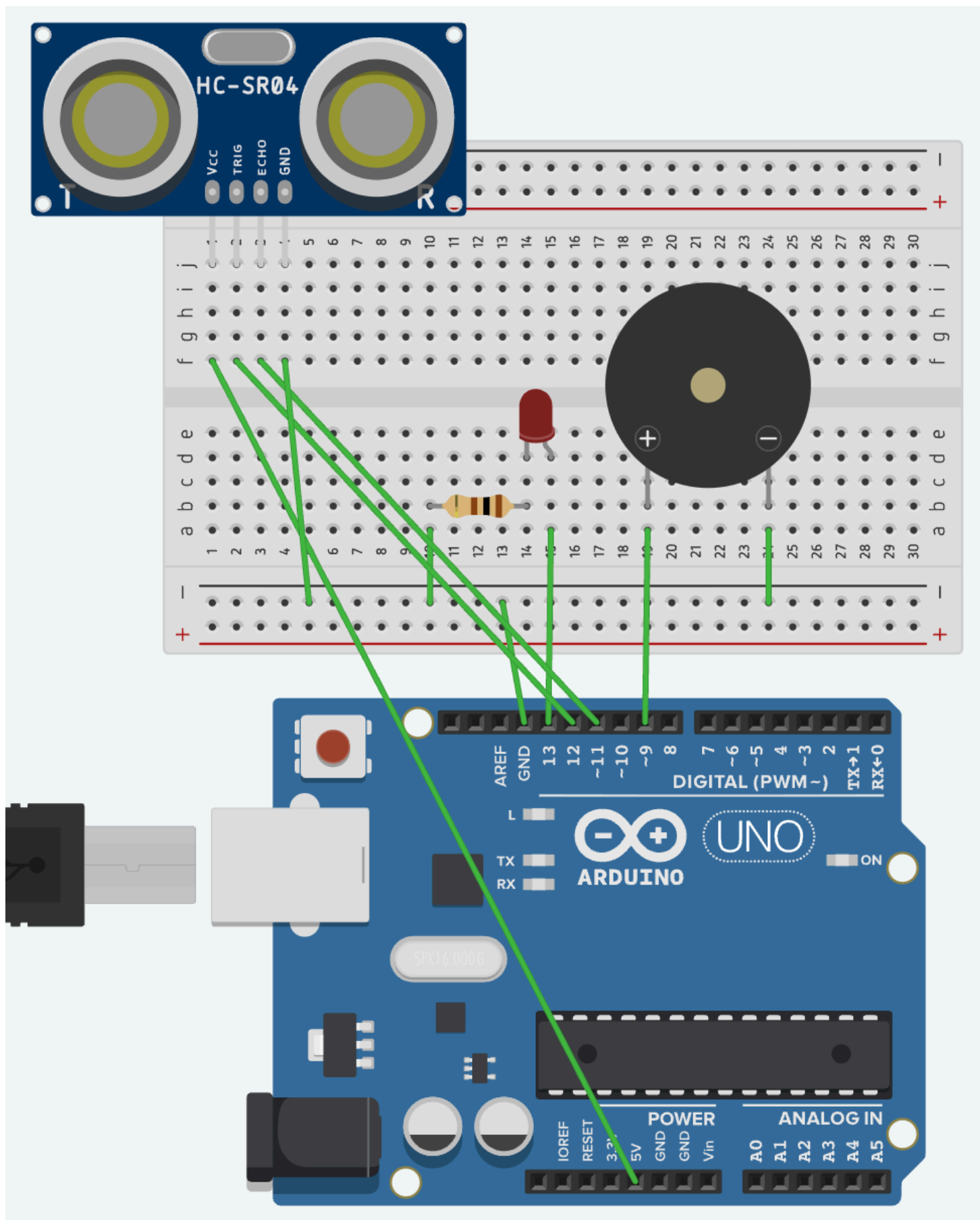
void loop() {
    //トリガーピンを一瞬HIGHにして、超音波を発信
    digitalWrite(trigPin, LOW);
    delayMicroseconds(5);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    //エコーピンがHIGHになった時間を測定する
    duration = pulseIn(echoPin, HIGH);

    //時間を距離に変換する
    //距離[cm] = ((音速[cm/s])*時間[s])/2
    //音速は約343m/s = 34300cm/s
    //durationはマイクロ秒単位なので、1[μs] = 0.000001[s]
    //音波は対象物まで往復しているので2で割る
    cm = duration * 0.000001 * 34000 / 2;

    Serial.print(cm); //センチメートルを出力する
    Serial.print("cm"); //センチメートルの単位を出力する
    Serial.println(); //改行

    delay(250);
}
```



書き込んだ後、Arduino IDE上部の虫眼鏡マークをクリックし、シリアルモニタを開くと超音波センサーからの距離が表示される。

シリアルモニタ



シミュレータ: <https://www.tinkercad.com/things/6BO0btq0w3n-sensor?sharecode=1PAincofvQBAYF26EJCTyZ1iDyQXMeGD2q2zsEvuALU>

シリアル通信を保存する (Windows)

[シリアルデータの受信](#)

シリアル通信を保存する (Mac)

[とにかく最速でSerial通信のログをファイルに残す](#)

センサーを使ったリアルタイム制御

超音波センサーが検知した距離に応じてLEDの明るさとスピーカーの音量を変更する。

```
//超音波センサー（HC-SR04）での距離を計測し、シリアルモニタに出力する
//検知したcmに応じてLEDの明るさとスピーカーの音量を変更する
//ただし、デューティ比は255を超えることはできない
```

```
// VCC → 5V
// GND → GND
// Trig → 12
// Echo → 11
```

```
int trigPin = 12; // Trigger
int echoPin = 11; // Echo
long duration, cm; //距離を格納する変数
```

```
void setup() {
    Serial.begin(9600); //シリアル通信を開始する
    pinMode(trigPin, OUTPUT); //トリガーピンを出力に設定する
    pinMode(echoPin, INPUT); //エコーピンを入力に設定する

    pinMode(13, OUTPUT); //LED 13ピンを出力に設定する
    pinMode(9, OUTPUT); //スピーカー 9ピンを出力に設定する
}
```

```
void loop() {
    //トリガーピンを一瞬HIGHにして、超音波を発信
    digitalWrite(trigPin, LOW);
    delayMicroseconds(10);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    //エコーピンがHIGHになっている時間を測定する
    duration = pulseIn(echoPin, HIGH);

    //時間を距離に変換する
    //距離[cm] = ((音速[cm/s])*時間[s])/2
    //音速は約343m/s = 34300cm/s
    //durationはマイクロ秒単位なので、1[μs] = 0.000001[s]
    //音波は対象物まで往復しているのので2で割る
    cm = duration * 0.000001 * 34000 / 2;

    Serial.print(cm); //センチメートルを出力する
    Serial.print("cm"); //センチメートルの単位を出力する
    Serial.println(); //改行
```

```
analogWrite(13, max(255, cm)); // 距離に応じてLEDの明るさを変更
analogWrite(9, max(255, cm)); // 距離に応じてスピーカーの音を変更
// analogWriteの引数は0から255なので、max関数で255を超えないようにしている

delay(250);

}
```

課題

いろいろなパラメータを変えてみる。

- 出力するパーツ(LED、スピーカー)
- 入力するパーツ(超音波センサー、ボタン)
- toneの周波数やデューティ比

参考になるWebページ

- Arduino言語日本語リファレンス <https://www.musashinodenpa.com/arduino/ref/>
- 一週間で身につくC言語の基本 <https://c-lang.sevendays-study.com/index.html>